

Autism Spectrum Disorder Screening with Machine Learning

From a leaky baseline to a foundation-model ensemble

Trần Trung Hiếu
Phạm Song Hà
Nguyễn Đức Hoàng
Vũ Xuân Sơn
Đặng Quốc Vượng

Hanoi University of Science and Technology

Ngày 26 tháng 6 năm 2026

Outline

- 1 Problem & Pipeline
- 2 Data & the leakage pitfall
- 3 Data generation
- 4 Models
- 5 Results
- 6 Conclusion

The problem

- Diagnosing ASD is **slow and costly**; early screening helps early support.
- The **AQ-10** questionnaire (A1–A10) gives a fast screening score.
- A fixed score cut-off is unreliable in the **grey zone** (score 4–6).

Goal. Learn from AQ-10 + demographics to flag likely ASD cases — **accurately** and **honestly** (no data leakage).

Project pipeline

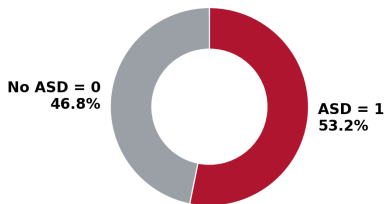


- One leakage-free cleaning pipeline feeds **every** model.
- Every model uses the **same 5-fold Stratified CV** — a fair race.

The raw data — and its traps

- 3,743 rows, 22 columns, **no missing values**.
- age max = **383** $\Rightarrow$ data-entry error.
- contry_of_res,
used_app_before: **one value only**
(no signal).
- ethnicity: duplicates from typos
(White European vs
White-European).

**Target is balanced
(SMOTE/GAN not needed for balance)**



Target is balanced (53/47) $\Rightarrow$ SMOTE/GAN are for *robustness*, not for balance.

Cleaning & encoding

- Drop zero-variance columns; merge typo categories.
- Clip the age outlier to the **median**.
- Encode categoricals with **One-Hot** — safe, no label peeking.

Output: a clean table reused by all later notebooks.

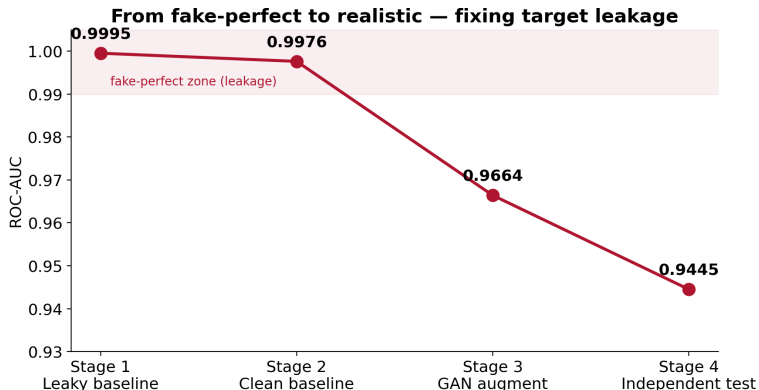
The target-leakage pitfall

Early baseline scored a **fake-perfect** ROC-AUC = 0.9995. Why?

$$\text{enc}(c) = \frac{1}{|\{i : x_i = c\}|} \sum_{i: x_i=c} y_i \quad \text{computed on **all** data before CV.}$$

- Encoding a category by its mean *label* leaks the answer y .
- The model “sees” the target through the feature $\Rightarrow$ unreal scores.
- **Fix:** compute encodings inside folds, or just use One-Hot.

Honest numbers beat perfect numbers



Score *drops* as we remove leakage and add realistic noise — this is the model getting **trustworthy**, not weaker.

SMOTE — interpolate new samples

Create a synthetic point between a sample and a neighbour:

$$x_{\text{new}} = x_i + \lambda (x_i^{\text{nn}} - x_i), \quad \lambda \sim U(0, 1).$$

- Fills gaps *between* real points (not blind copies).
- Applied **only on the training fold** to avoid leakage.

Tabular GAN (PyTorch, from scratch)

Two networks play a **min-max** game:

$$\min_G \max_D \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log (1 - D(G(z)))] .$$

- **Generator** G : turns noise z into fake patient rows.
- **Discriminator** D : tells real from fake.
- We sampled **+20k** train and **+2k** test rows $\Rightarrow$ harder, more realistic data.

How we compare

- **5-fold Stratified CV** on the unified **23,743**-row set.
- Headline metrics: **F1-Score** and **ROC-AUC** (threshold-free).
- We keep **one leaderboard** and add each new method to it.

Baselines — the classics

- **Logistic Regression** — linear log-odds: $\log \frac{p}{1-p} = w^\top x + b$. Simple, interpretable.
- **Random Forest** — average many de-correlated deep trees (bagging) to cut variance.
- **AdaBoost** — add stumps sequentially, re-weighting the samples each one gets wrong.
- **XGBoost** — regularized gradient boosting; our strongest baseline.

These set the bar the modern methods must beat.

Leaderboard — baselines

Starting point: **XGBoost** leads the four classics.

Model	Acc	F1	ROC-AUC
XGBoost	0.9162	0.9197	0.9767
Random Forest	0.9151	0.9191	0.9748
AdaBoost	0.8624	0.8656	0.9511
Logistic Regression	0.8643	0.8687	0.9473

CatBoost — ordered boosting

- Gradient boosting on **oblivious trees** (same split per level) — fast and regularized.
- Encodes categories with **ordered target statistics**, using only *earlier* rows:

$$\hat{x}_i = \frac{\sum_{j < i} \mathbb{I}[x_j = x_i] y_j + a p}{\sum_{j < i} \mathbb{I}[x_j = x_i] + a}.$$

- **Ordered boosting** stops a sample seeing its own label $\Rightarrow$ no prediction shift.

Directly cures the Phase-1 leakage — and trains in just 15.8s.

Leaderboard — + CatBoost

CatBoost takes over **#1**.

Model	Acc	F1	ROC-AUC
CatBoost	0.9207	0.9237	0.9794
XGBoost	0.9162	0.9197	0.9767
Random Forest	0.9151	0.9191	0.9748
AdaBoost	0.8624	0.8656	0.9511
Logistic Regression	0.8643	0.8687	0.9473

TabM — one MLP, many members

- A neural net (MLP), but it behaves like **k ensembled MLPs** at $\approx 1\times$ the cost.
- **BatchEnsemble**: one shared matrix W + a rank-1 adapter (r_k, s_k) per member:

$$W_k = W \odot (r_k s_k^\top), \quad \hat{p} = \frac{1}{k} \sum_{m=1}^k \text{softmax}(f_{W_m}(x)).$$

- Averaging members **reduces variance** (better calibration), adapters keep it cheap.

Top-tier accuracy (AUC 0.9796) but **slow to train** (~ 1637 s on CPU).

Leaderboard — + TabM

TabM edges ahead on ROC-AUC.

Model	Acc	F1	ROC-AUC
TabM	0.9203	0.9237	0.9796
CatBoost	0.9207	0.9237	0.9794
XGBoost	0.9162	0.9197	0.9767
Random Forest	0.9151	0.9191	0.9748
AdaBoost	0.8624	0.8656	0.9511
Logistic Regression	0.8643	0.8687	0.9473

LightGBM — histogram boosting

- Gradient boosting that **bins** continuous features into histograms $\Rightarrow$ fast split search.
- **Leaf-wise** (best-first) tree growth + sample/feature bundling for extra speed.
- Same additive idea, each tree fitting the negative gradient:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad h_m \approx -\nabla_F L.$$

The **fastest** learner here — a full 5-fold run in just 1.2s.

Leaderboard — + LightGBM

Fast and competitive — lands just behind the leaders.

Model	Acc	F1	ROC-AUC
TabM	0.9203	0.9237	0.9796
CatBoost	0.9207	0.9237	0.9794
LightGBM	0.9187	0.9220	0.9780
XGBoost	0.9162	0.9197	0.9767
Random Forest	0.9151	0.9191	0.9748
AdaBoost	0.8624	0.8656	0.9511
Logistic Regression	0.8643	0.8687	0.9473

TabPFN — a tabular foundation model

- A **transformer** pre-trained *once* on millions of synthetic tables.
- At use time: feed the training rows + a test row as context, predict in **one forward pass — no gradient training, no tuning**:

$$p(y \mid x, \mathcal{D}_{\text{train}}) \approx \text{single in-context inference.}$$

- Learns a prior over datasets $\Rightarrow$ approximates the Bayesian posterior predictive.

Best single model (F1 0.9275, AUC 0.9810) in ~ 42 s, straight out of the box.

Leaderboard — + TabPFN

TabPFN jumps straight to **#1** — with no tuning at all.

Model	Acc	F1	ROC-AUC
TabPFN	0.9241	0.9275	0.9810
TabM	0.9203	0.9237	0.9796
CatBoost	0.9207	0.9237	0.9794
LightGBM	0.9187	0.9220	0.9780
XGBoost	0.9162	0.9197	0.9767
Random Forest	0.9151	0.9191	0.9748
AdaBoost	0.8624	0.8656	0.9511
Logistic Regression	0.8643	0.8687	0.9473

Stacking & blending

- Train base models with CV, keep their **out-of-fold (OOF)** predictions (leakage-free).
- **Blending** — average the probabilities: $\hat{p} = \frac{1}{M} \sum_m p_m(x)$.
- **Stacking** — a logistic-regression meta-learner over those predictions:

$$\hat{p} = \sigma\left(\sum_m w_m p_m(x) + b\right).$$

- Diverse models' errors partly cancel $\Rightarrow$ **steadier** than any single model.

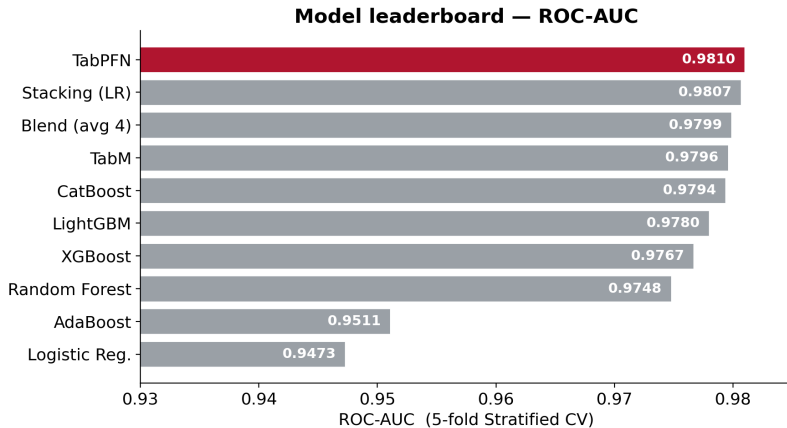
Base learners: TabPFN + CatBoost + LightGBM + XGBoost.

Leaderboard — + Stacking & Blending

Ensembles fill the podium just under TabPFN.

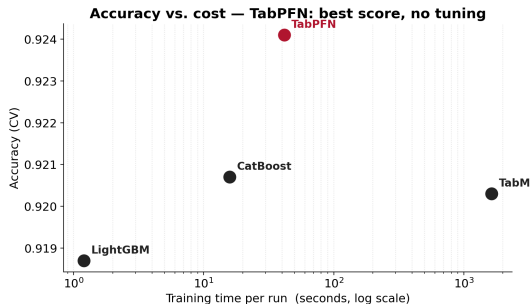
Model	Acc	F1	ROC-AUC
TabPFN	0.9241	0.9275	0.9810
Stacking (LR)	0.9232	0.9266	0.9807
Blend (avg 4)	0.9226	0.9258	0.9799
TabM	0.9203	0.9237	0.9796
CatBoost	0.9207	0.9237	0.9794
LightGBM	0.9187	0.9220	0.9780
XGBoost	0.9162	0.9197	0.9767
Random Forest	0.9151	0.9191	0.9748
AdaBoost	0.8624	0.8656	0.9511
Logistic Regression	0.8643	0.8687	0.9473

Final leaderboard



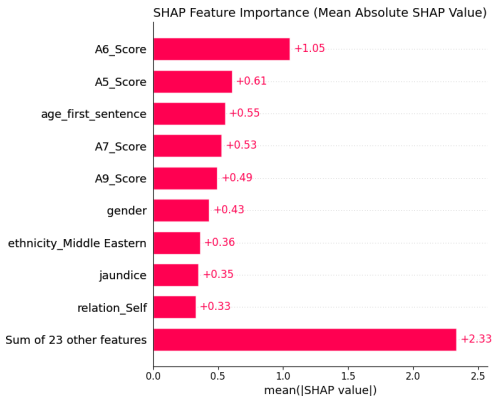
From **LogReg 0.947** to **TabPFN 0.981** ROC-AUC. Top-3 = foundation model + ensembles; a zero-tuning model wins.

Accuracy vs. cost



- **LightGBM**: 1.2s, very strong value.
- **TabPFN**: best score, ~42s, no tuning.
- **TabM**: 1637s — accuracy at a steep cost.

Explainable AI — which features matter (global)

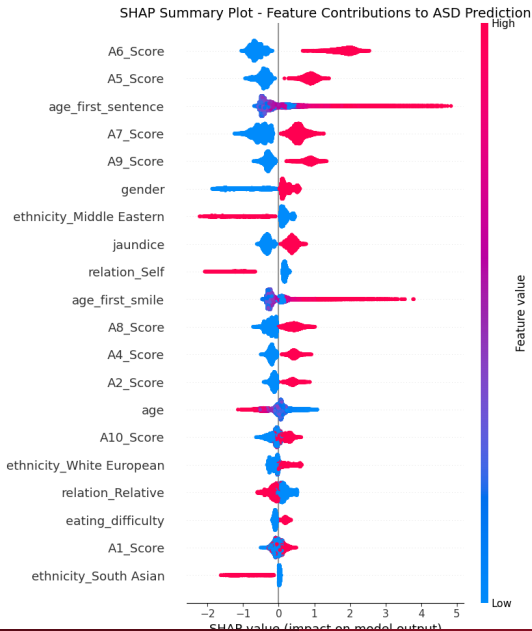


SHAP value = a feature's fair contribution (Shapley value):

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} (f(S \cup \{i\}) - f(S))$$

- **AQ-10 items** (A6, A5, A7, A9) on top.
- **Developmental milestones** (age_first_sentence) just as strong.

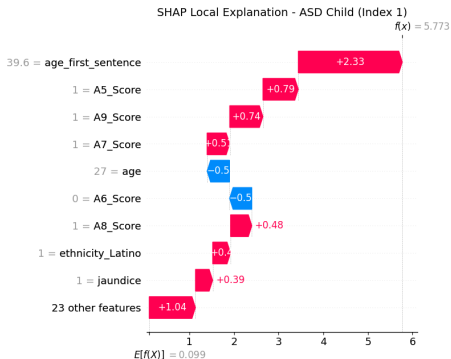
SHAP — how each feature pushes the output



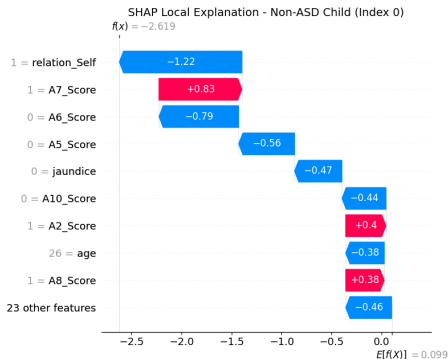
- One dot = one patient;
colour = feature value (red high, blue low).
- Right of 0 $\Rightarrow$ pushes towards **ASD**.
- **High** AQ-10 scores (red, right) raise ASD risk — clinically sensible.
- Late
age_first_sentence (red) strongly raises risk.

SHAP — per-patient explanations

ASD case $f(x) = 5.77$



Non-ASD case $f(x) = -2.62$



Red bars push up (towards ASD), blue down. Same model, a **transparent reason** for every individual prediction.

Conclusion & future work

- ① Built a **leakage-free** pipeline — honest scores over fake-perfect ones.
- ② **GAN/SMOTE** augmentation made the model robust to realistic noise.
- ③ Fair race of 10 models: **TabPFN** wins; **stacking** is a steady runner-up.
- ④ **SHAP** confirms clinically sensible drivers.

Next: deploy as a phone-based pre-screening aid for parents.

References I



Kaggle, *Autism Prediction Challenge* dataset.



Chawla et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR.



Goodfellow et al. (2014). *Generative Adversarial Networks*. arXiv:1406.2661.



Chen & Guestrin (2016). *XGBoost: A Scalable Tree Boosting System*.
arXiv:1603.02754.



Ke et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*.
NeurIPS.



Prokhorenkova et al. (2018). *CatBoost: Unbiased Boosting with Categorical Features*. arXiv:1706.09516.



Lundberg & Lee (2017). *A Unified Approach to Interpreting Model Predictions (SHAP)*. arXiv:1705.07874.



Hollmann et al. (2022). *TabPFN: A Transformer That Solves Small Tabular Problems in a Second*. arXiv:2207.01848.



Gorishniy et al. (2024). *TabM: Advancing Tabular DL with Parameter-Efficient Ensembling*. arXiv:2410.24210.

Thank you!

Questions & discussion